

R Programming: Application-Level

Problem Statements for Internal POE

Big Data Analytics — B.Tech Program

This document contains a curated list of R programming and Big Data infrastructure problems designed for B.Tech students. Each problem is structured to test specific core competencies including data structures, manipulation, visualization, statistical modeling, and distributed computing environments.

1. Sales Performance Analysis

Focus: Vectors & Basic Plotting

Problem: A retail store wants to analyze the weekly sales of two different branches. Create two vectors representing daily sales for 7 days. Calculate the total weekly sales, the average daily sale, and visualize the comparison using a Bar Chart.

```
# Data Creation
branch_A <- c(1200, 1500, 1100, 1800, 2000, 2500, 2100)
branch_B <- c(1000, 1300, 1400, 1600, 1900, 2200, 2400)

# Calculations
total_A <- sum(branch_A)
avg_A <- mean(branch_A)

# Visualization
barplot(rbind(branch_A, branch_B), beside = TRUE,
        col = c("blue", "green"),
        names.arg = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"),
        main = "Weekly Sales Comparison", xlab = "Day", ylab = "Sales ($)")
legend("topleft", legend = c("Branch A", "Branch B"), fill = c("blue", "green"))
```

2. Multi-Dimensional Sensor Data

Focus: Arrays

Problem: A weather station records temperature and humidity across two different zones over four time intervals. Store this data in a 3D Array. Calculate the mean temperature for each zone across all time intervals.

```
# Creating a 2x4x2 array (2 Zones, 4 Time slots, 2 Metrics: Temp & Humidity)
data_values <- c(22, 24, 21, 25, 30, 31, 29, 32, 60, 65, 70, 75, 55, 50, 58, 62)
weather_array <- array(data_values, dim = c(2, 4, 2),
                       dimnames = list(c("Zone1", "Zone2"),
                                       c("T1", "T2", "T3", "T4"),
                                       c("Temp", "Humidity")))

# Mean Temperature for Zone 1
mean_zone1_temp <- mean(weather_array["Zone1", , "Temp"])
print(weather_array)
print(paste("Mean Temp Zone 1:", mean_zone1_temp))
```

3. Student Grade Frequency

Focus: R Tables & Pie Charts

Problem: Given a list of grades for 50 students, generate a Frequency Table to show how many students received each grade (A, B, C, D, F). Represent the distribution using a Pie Chart.

```
# Generating random grade data
grades <- sample(c("A", "B", "C", "D", "F"), 50, replace = TRUE)

# Frequency Table
grade_table <- table(grades)
print(grade_table)

# Pie Chart
pie(grade_table, main = "Grade Distribution", col = rainbow(5))
```

4. Employee Database Management

Focus: Data Manipulation & Shaping

Problem: Create a data frame containing Employee ID, Name, and Department. Filter the data to show only employees in the "IT" department and Sort the resulting list by Name. Use basic R to transform data from wide to long format.

```
# Data Frame
emp_data <- data.frame(ID = 1:4,
                      Name = c("Alice", "Bob", "Charlie", "David"),
                      Dept = c("IT", "HR", "IT", "Finance"),
                      Salary = c(50000, 45000, 55000, 60000))

# Filtering and Sorting
it_employees <- emp_data[emp_data$Dept == "IT", ]
sorted_it <- it_employees[order(it_employees$Name), ]

# Shaping: Wide to Long using reshape
long_data <- reshape(emp_data, direction="long", varying=list(4),
                    v.names="Value", times="Salary", idvar="ID")

print(sorted_it)
print(head(long_data))
```

5. Healthcare Record Merging

Focus: Merging & Data Structures

Problem: You have two datasets: one with Patient ID and Personal Info, and another with Patient ID and Medical Diagnosis. Merge these datasets into a single data frame and identify any missing records.

```
# Datasets
personal <- data.frame(PID = c(101, 102, 103), Name = c("John", "Jane", "Doe"))
medical <- data.frame(PID = c(101, 103, 104), Diagnosis = c("Flu", "Cold", "Fever"))

# Inner Merge
complete_records <- merge(personal, medical, by = "PID")

# Full Merge (includes NAs for missing info)
all_records <- merge(personal, medical, by = "PID", all = TRUE)
print(all_records)
```

6. Population Growth Distribution

Focus: Histograms

Problem: Simulate a dataset of 100 city populations. Create a Histogram to visualize the distribution frequency of population sizes and add a density line to the plot.

```
# Data Stream Simulation
set.seed(123)
populations <- rnorm(100, mean = 500000, sd = 100000)

# Histogram
hist(populations, breaks = 10, col = "orange", border = "black",
     main = "City Population Distribution", xlab = "Population Count")
```

7. Predicting House Prices

Focus: Linear Regression

Problem: Given a dataset of House Size (sq ft) and Price, build a Linear Regression model to predict the price of a house. Plot the data points and the regression line.

```
# Data
sqft <- c(1500, 1800, 2400, 3000, 3500)
price <- c(300000, 350000, 450000, 500000, 600000)

# Model Building
model <- lm(price ~ sqft)
summary(model)

# Plotting
plot(sqft, price, main = "Price vs Size", pch = 19, col = "blue")
abline(model, col = "red", lwd = 2)
```

8. Real-time Log Monitoring

Focus: Data Stream Simulation

Problem: Simulate a continuous Data Stream of CPU usage percentages. Write a loop that processes input, calculates a running average every 10 steps, and stops if the usage exceeds 95%.

```
# Simulating a stream loop
set.seed(42)
stream_buffer <- c()
for(i in 1:100) {
  current_usage <- runif(1, 40, 100) # Simulating input
  stream_buffer <- c(stream_buffer, current_usage)
  if(i %% 10 == 0) {
    cat("Batch", i/10, "- Average Usage:", mean(stream_buffer), "\n")
  }
  if(current_usage > 95) {
    print(paste("Critical Alert! Usage reached:", round(current_usage, 2)))
    break
  }
}
```

9. Installation of Hadoop

Focus: Big Data Infrastructure — Distributed Storage & Processing

Problem: Install and configure Apache Hadoop (Single-Node / Pseudo-Distributed Mode) on Ubuntu. Verify the installation by running a MapReduce word-count job on a sample text file.

Step 1 — Prerequisites

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install Java (Hadoop requires Java 8 or 11)
sudo apt install -y openjdk-11-jdk

# Verify Java installation
java -version

# Install SSH (required for Hadoop daemon communication)
sudo apt install -y ssh
sudo systemctl enable ssh && sudo systemctl start ssh

# Generate SSH key and add to authorized_keys (passwordless login)
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost    # Should connect without password prompt
```

Step 2 — Download & Install Hadoop

```
# Download Hadoop 3.3.6 (stable release)
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

# Extract and move to /usr/local
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop

# Set ownership
sudo chown -R $USER:$USER /usr/local/hadoop
```

Step 3 — Configure Environment Variables

```
# Add to ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME

source ~/.bashrc
hadoop version    # Verify
```

Step 4 — Configure Hadoop XML Files

Edit the following core configuration files inside `/usr/local/hadoop/etc/hadoop/`:

```
# --- core-site.xml ---
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>

# --- hdfs-site.xml ---
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
```

Step 5 — Format HDFS & Start Hadoop

```
# Format the Hadoop filesystem (only once on first setup)
hdfs namenode -format

# Start all Hadoop services
start-dfs.sh
start-yarn.sh

# Verify running daemons
jps
# Expected output: NameNode, DataNode, ResourceManager, NodeManager

# Access Hadoop Web UI
# NameNode:      http://localhost:9870
# ResourceMgr:   http://localhost:8088
```

Step 6 — Run a Sample MapReduce Job

```
# Create HDFS input directory and upload a sample file
hdfs dfs -mkdir -p /user/$USER/input
echo 'Hello Hadoop Big Data World' > sample.txt
hdfs dfs -put sample.txt /user/$USER/input/

# Run the built-in WordCount example
hadoop jar /usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
  wordcount /user/$USER/input /user/$USER/output

# View results
hdfs dfs -cat /user/$USER/output/part-r-00000
```

10. Installation of Apache Spark

Focus: Big Data Infrastructure — In-Memory Processing

Problem: Install Apache Spark in Standalone mode on Ubuntu. Write a basic PySpark script to read a CSV file, perform a word count, and display the results.

Step 1 — Prerequisites

```
# Java must already be installed (see Q9)
java -version

# Install Python 3 and pip
sudo apt install -y python3 python3-pip

# Install PySpark via pip (includes Spark binaries)
pip3 install pyspark

# Verify
pyspark --version
```

Step 2 — Download & Install Standalone Spark (Optional — for Cluster Mode)

```
# Download Spark 3.5.1
wget https://downloads.apache.org/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
tar -xzf spark-3.5.1-bin-hadoop3.tgz
sudo mv spark-3.5.1-bin-hadoop3 /usr/local/spark

# Add to ~/.bashrc
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
source ~/.bashrc

# Start Spark Master
start-master.sh

# Access UI: http://localhost:8080
```

Step 3 — PySpark Word Count Script

```
# wordcount.py
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("WordCount") \
    .getOrCreate()

# Read a text file into an RDD
rdd = spark.sparkContext.textFile("sample.txt")

# Perform Word Count
counts = (rdd
    .flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b))

# Show results
for word, count in counts.collect():
    print(f"{word}: {count}")

spark.stop()
```

Step 4 — Run the Script

```
# Submit the PySpark job
spark-submit wordcount.py

# Or run interactively in PySpark shell
pyspark
>>> rdd = sc.textFile('sample.txt')
>>> rdd.flatMap(lambda x: x.split()).map(lambda w: (w,1)).reduceByKey(lambda
a,b:a+b).collect()
```

11. Installation of Hive via Docker

Focus: Big Data Infrastructure — Data Warehousing

Problem: Set up Apache Hive using Docker and Docker Compose. Connect to the Hive server, create a database, define an external table, and run basic HiveQL queries.

Step 1 — Install Docker & Docker Compose

```
# Install Docker
sudo apt update
sudo apt install -y docker.io docker-compose

# Start and enable Docker service
sudo systemctl start docker
sudo systemctl enable docker

# Add current user to docker group (avoid using sudo each time)
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker-compose --version
```

Step 2 — Create docker-compose.yml for Hive

```
# docker-compose.yml
version: '3.8'
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    environment:
      - CLUSTER_NAME=hive-cluster
    ports:
      - '9870:9870'
      - '9000:9000'
    volumes:
      - hadoop_namenode:/hadoop/dfs/name

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    depends_on: [namenode]
    volumes:
      - hadoop_datanode:/hadoop/dfs/data
```

```

hive-server:
  image: bde2020/hive:2.3.2-postgresql-metastore
  container_name: hive-server
  depends_on: [namenode, datanode]
  ports:
    - '10000:10000'
    - '10002:10002'
  environment:
    -
HIVE_CORE_CONF_javax_jdo_option_ConnectionURL=jdbc:postgresql://hive-metastore/metastore

hive-metastore:
  image: bde2020/hive:2.3.2-postgresql-metastore
  container_name: hive-metastore
  command: /opt/hive/bin/hive --service metastore

hive-metastore-postgresql:
  image: bde2020/hive-metastore-postgresql:2.3.0
  container_name: hive-metastore-postgresql

volumes:
  hadoop_namenode:
  hadoop_datanode:

```

Step 3 — Start the Hive Cluster

```

# Launch all containers in detached mode
docker-compose up -d

# Check all containers are running
docker ps

# Wait ~60 seconds for services to initialize, then connect to Hive
docker exec -it hive-server /bin/bash
beeline -u jdbc:hive2://localhost:10000

```

Step 4 — Run HiveQL Queries

```

-- Create a new Hive database
CREATE DATABASE IF NOT EXISTS bigdata_db;
USE bigdata_db;

-- Create a managed table
CREATE TABLE students (
  student_id INT,
  name       STRING,
  grade      STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Load data into the table
LOAD DATA LOCAL INPATH '/path/to/grades.csv' INTO TABLE students;

-- Query the data
SELECT * FROM students LIMIT 10;
SELECT grade, COUNT(*) AS total FROM students GROUP BY grade ORDER BY grade;

```


Step 5 — Stop the Cluster

```
# Stop and remove all containers
docker-compose down

# To also remove volumes (clears all data)
docker-compose down -v
```

12. Basic Docker Script for Student Grade Table

Focus: Docker — Containerized Database Workflow

Problem: Write a complete Docker-based workflow to create a MySQL container, define a `student_grades` table, insert sample data, and run SQL queries to analyze grade distribution — all using Docker commands and SQL scripts.

Step 1 — Pull and Run a MySQL Container

```
# Pull the official MySQL 8.0 image
docker pull mysql:8.0

# Run MySQL container with root password and a named database
docker run -d \
  --name student-db \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -e MYSQL_DATABASE=school_db \
  -p 3306:3306 \
  mysql:8.0

# Verify the container is running
docker ps

# Wait for MySQL to fully start (~15 seconds), then connect
docker exec -it student-db mysql -u root -prootpass school_db
```

Step 2 — SQL Script: Create Table & Insert Data

```
-- Create the student_grades table
CREATE TABLE IF NOT EXISTS student_grades (
  student_id INT NOT NULL AUTO_INCREMENT,
  student_name VARCHAR(100) NOT NULL,
  subject VARCHAR(50) NOT NULL,
  marks INT NOT NULL,
  grade CHAR(1) NOT NULL,
  PRIMARY KEY (student_id)
);

-- Insert sample data
INSERT INTO student_grades (student_name, subject, marks, grade) VALUES
  ('Alice', 'Big Data Analytics', 92, 'A'),
  ('Bob', 'Big Data Analytics', 76, 'B'),
  ('Charlie', 'Big Data Analytics', 65, 'C'),
  ('Diana', 'Big Data Analytics', 88, 'A'),
  ('Eve', 'Big Data Analytics', 54, 'D'),
  ('Frank', 'Big Data Analytics', 91, 'A'),
  ('Grace', 'Big Data Analytics', 73, 'B'),
  ('Henry', 'Big Data Analytics', 48, 'F'),
```

```
('Iris',      'Big Data Analytics', 85, 'A'),
('Jack',      'Big Data Analytics', 60, 'C');
```

Step 3 — SQL Queries for Grade Analysis

```
-- View all student records
SELECT * FROM student_grades ORDER BY student_name;

-- Grade frequency distribution
SELECT grade,
       COUNT(*) AS num_students,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM student_grades), 1) AS percentage
FROM student_grades
GROUP BY grade
ORDER BY grade;

-- Average marks per grade
SELECT grade, ROUND(AVG(marks), 2) AS avg_marks
FROM student_grades
GROUP BY grade
ORDER BY avg_marks DESC;

-- Students scoring above average
SELECT student_name, marks, grade
FROM student_grades
WHERE marks > (SELECT AVG(marks) FROM student_grades)
ORDER BY marks DESC;

-- Summary statistics
SELECT
    COUNT(*)           AS total_students,
    MAX(marks)          AS highest_marks,
    MIN(marks)          AS lowest_marks,
    ROUND(AVG(marks), 2) AS average_marks
FROM student_grades;
```

Step 4 — Run SQL Script via Docker (Non-Interactive)

```
# Save the SQL as grades_setup.sql, then run it directly:
docker exec -i student-db mysql -u root -prootpass school_db < grades_setup.sql

# Or run a single inline query:
docker exec -it student-db mysql -u root -prootpass school_db \
    -e "SELECT grade, COUNT(*) AS count FROM student_grades GROUP BY grade;"
```

Step 5 — Stop & Clean Up

```
# Stop the container
docker stop student-db

# Remove the container
docker rm student-db

# (Optional) Remove the MySQL image
docker rmi mysql:8.0
```

